# Appendix C

# Agentic Autoresearch for Recommender Systems

After reading this appendix, you will be able to:

- Separate the mechanical steps of recommender research (edit, run, score, record) from the judgment steps (choose the question, define the label and split, decide what counts as evidence), and say which steps an agent can own.
- Explain how repeated validation queries, a visible or editable scorer, and reachable test data each turn an automated search into a source of false improvements, and recognize the traces each leaves in the experiment records.
- Specify a harness contract: which artifacts are sealed, which code is mutable, who may change the seal, and how a new harness version is created.
- Design a promotion gate that recomputes metrics in the harness, checks leakage and coverage, requires a gain beyond the noise band, records every trial including failures, and spends sealed test data once.
- Decide from evaluator quality, trial cost, and review capacity whether an agentic loop is worth building, or whether a scripted sweep or random search under the same harness is the better tool.
- Run a guarded search loop on the Chapter 6 ranker on a laptop, and interpret a positive or a negative result.

## C.1 Why Automate the Research Loop

Most of a recommender team's research time goes into one loop: change something in the training recipe, train, score the result on held-out data, compare it with what came before, and decide what to try next. A coding agent can now run much of that loop. A coding agent is a program in which a large language model (Chapter 12) reads files, writes code, runs commands, and reads their output, repeating until a task is done or a budget runs out. Several industrial teams have published systems in which agents propose, implement, and evaluate changes to production ranking models (Kumar et al., 2026; Gao et al., 2026; Wang et al., 2026; AgentX Team, 2026), and automated search over click-through-rate model architectures predates language-model agents (Song et al., 2020).

This appendix covers agents that build and improve a recommender offline. Agents that answer user requests at serving time are a different subject, and Section 12.8.6 covers them while pointing here for agents that automate recommender research and experimentation. The agents in this appendix never talk to users. They change code and configuration, and their output is a candidate model with a record of how it was found.

### C.1.1 The Research Loop of a Recommender Team

Table C.1 walks the loop on the pointwise XGBoost ranker of Chapter 6, which predicts whether a user will play at least half of a track. Each step is marked as mechanical, meaning it can be specified completely and checked by code, or as a judgment step, which needs someone to decide what the result means or what should count as evidence.

*Table C.1: The recommender research loop on the Chapter 6 ranker.*

| Step | Example on the Chapter 6 ranker | Mechanical or judgment step | Owner in the guarded loop | What checks it |
|---|---|---|---|---|
| Choose the question | Does a shorter training lookback lose accuracy? | Judgment | Human owner, with a research agent drafting | Review of the written brief |
| Define label, split, metric | Positive when at least half the track is played; train, gap, one-day validation, gap, one-day test; AUC as the primary metric | Judgment | Harness Auditor, signed off by the human owner | Sealed contract and file hashes |
| Propose a change | Add an aggregate feature; change tree depth | Mixed | Research agent or scripted proposer | Critic review against the ledger |
| Implement | Edit the training script in a private workspace | Mechanical | Implementation agent | Leakage checks and Critic review |
| Run | Train under a fixed time and memory budget | Mechanical | A runner outside the agent's control | Runner limits and status codes |
| Evaluate | Score validation predictions | Mechanical | Sealed evaluator | Hash check; metrics recomputed from predictions |
| Interpret | Is the gain larger than seed and resampling noise? Why did it help? | Judgment | Critic, then the human owner | Noise band; ablations |

| Step | Example on the Chapter 6 ranker | Mechanical or judgment step | Owner in the guarded loop | What checks it |
|---|---|---|---|---|
| Decide promotion | Spend one sealed-test evaluation on this candidate? | Judgment | Harness Auditor, signed off by the human owner | Promotion gate (Section C.5) |
| Record | Append the trial, failed runs included | Mechanical | Ledger code | Schema validation; trial count |

The mechanical rows are where agents save time. The judgment rows decide whether any of that time produced knowledge. An agent lowers the cost of a trial, while the cost of a claim someone can trust is still set by evaluation and review, so automation moves the bottleneck from implementation to checking. Recommender offline metrics are also noisy relative to the gains teams chase (Chapter 3), and a loop that compares hundreds of candidates on one validation window will find some that look better by chance. Section C.2 shows how.

### C.1.2 Autoresearch, Defined

"Autoresearch" is an informal term, popularized by an open-source repository published in March 2026 (Karpathy, 2026). In that repository, one file prepares the data and evaluates the model and is never modified. The agent may edit only the training file. A human edits a plain-text instruction file that tells the agent what to explore. Every run gets the same fixed time budget, and the agent keeps a change when the validation metric improves and discards it otherwise.

This appendix uses the term for any loop in which an agent edits experiment code, runs it under a fixed budget, scores it with a fixed evaluator, and keeps or discards the change. The property that makes the minimal design safe is easy to miss: the evaluator sits outside the files the agent can edit. Everything in Sections C.3 to C.6 elaborates that one property for a recommender, where the evaluation also involves a temporal split, a label definition, leakage rules, and a test set that must stay unused.

### C.1.3 Where the Loop Sits

Figure C.1 places the loop in a recommender's lifecycle. The loop's output is a candidate that has passed a research promotion gate and one sealed-test evaluation. That candidate then enters the release ladder of Chapter 15, whose offline validation gate in Section 15.9.5 applies to candidates from automated research loops exactly as it does to scheduled retraining. After that comes the evidence ladder of Chapter 16, where, as Section 16.10.3 puts it, offline success from an automated loop earns a candidate online traffic and nothing more.

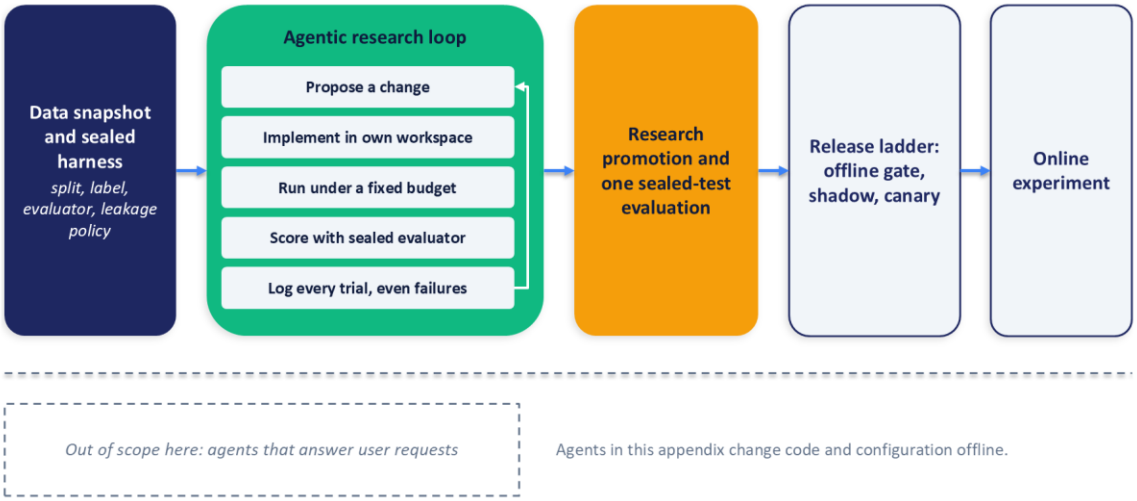



*Figure C.1: The agentic research loop sits between a sealed evaluation harness and the release ladder. It ends in research promotion and one sealed-test evaluation, and its output is a candidate for Chapter 15's offline gate, shadow, and canary stages and Chapter 16's online experiments. Agents that answer user requests, covered in Chapter 12, are out of scope.*

### C.1.4 Two Meanings of "Harness"

This appendix uses "evaluation harness", or "sealed harness", for a frozen evaluation contract and the code that enforces it, defined fully in Section C.3. Several industrial reports use "harness" for something else: the prompts, tools, and protocols wrapped around the language model that drives an agent (AgentX Team, 2026; Gao et al., 2026). To keep the two apart, this appendix calls that second object the agent scaffold. The distinction matters because the scaffold is something a team improves over time, sometimes automatically, whereas the evaluation harness is the part nobody inside the loop may change.

## C.2 How Automated Search Fools You

An automated loop inherits every way a human researcher can fool themselves, and it adds speed and volume, so it exploits whatever the evaluation allows. This section names the mechanisms before any remedy, so that each guard in Sections C.3 to C.6 answers a specific problem.

### C.2.1 The Best-of-N Illusion

A validation score is a noisy estimate of a candidate's true quality, and selecting the best of many noisy estimates selects noise along with quality. Section 3.5.1 warned against making design decisions on the validation set and then reporting results on the same set. An agent turns that warning into a quantitative problem, because it makes each additional query nearly free. The effect has been studied from several directions. Choosing the hypothesis with the lowest cross-validation error among many candidates overfits the cross-validation data (Ng, 1997), and model selection in general can overfit its selection criterion (Cawley and Talbot, 2010). When each new analysis depends on the results of earlier queries to the same holdout, the statistical guarantees that justify a holdout no longer hold (Dwork et al., 2015), and a public leaderboard can be overfit through its score feedback alone (Blum and Hardt, 2015).

A small model makes the size of the effect concrete. Let candidate $k$ have true quality $q_k$, validation score $v_k = q_k + \varepsilon_k$, and test score $t_k = q_k + \delta_k$, where $q_k$, $\varepsilon_k$, and $\delta_k$ are independent and normally distributed with mean zero and standard deviations $\sigma_q$, $\sigma_\varepsilon$, and $\sigma_\delta$. A loop that compares $K$ candidates and keeps the one with the highest validation score, $k^*$, reports $v_{k^*}$. The best estimate of that candidate's true quality, and so of its expected test score, shrinks the validation score toward the average:

$$\mathbb{E}[t_{k^*} \mid v_{k^*}] = \frac{\sigma_q^2}{\sigma_q^2 + \sigma_\varepsilon^2} v_{k^*}$$

When candidates differ little relative to validation noise, the shrinkage factor is small and most of the reported gain disappears on fresh data. Figure C.2 shows the effect in a seeded simulation in which the spread of true quality is three tenths of the validation noise, so the shrinkage factor is below one tenth.

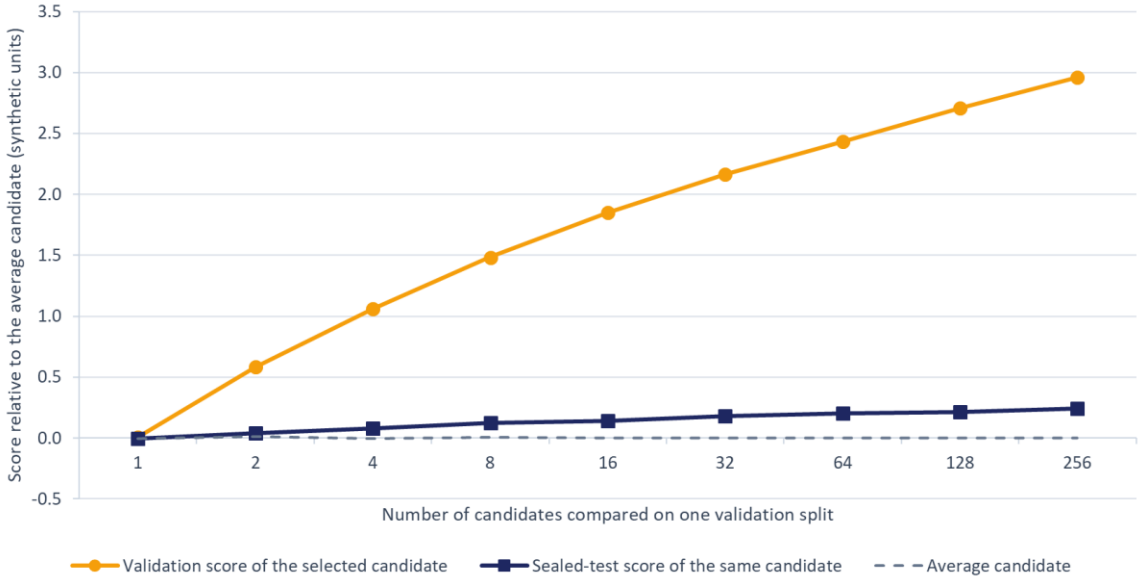



*Figure C.2: The best-of-N illusion in a seeded synthetic simulation. Candidate quality is normal with standard deviation 0.3, and validation and test scores each add independent standard normal noise. As more candidates are compared on one validation split, the selected candidate's validation score climbs steeply while its sealed-test score rises far less. Units are synthetic, relative to the average candidate.*

There is counter-evidence, and it sets the scope of the worry. A meta-analysis of 120 Kaggle competitions, where teams repeatedly scored submissions against a public holdout and were ranked once on a private one, found little evidence of substantial adaptive overfitting; the clear exceptions had very small test sets or splits that were not identically distributed (Roelofs et al., 2019). A one-day temporal validation window is a shifted, non-identically distributed sample by construction, which is the regime the study flags, and an agent loop issues more queries with more freedom to change code than a competition entrant does. No published study compares the overfitting of agent loops with that of human teams, so the risk here is an inference from theory and query volume, which the exercise in Section C.8 lets you test.

### C.2.2 When the Scorer Is Visible

The second mechanism appears when the agent can see or edit the scoring code. METR reports frontier models modifying tests or scoring code and monkey-patching evaluators to obtain high scores, and

found it far more often on a task suite in which the model could see the entire scoring function, a difference the authors suggest visibility may explain while noting that the suites also differ in other ways (Von Arx et al., 2025). In separate runs, an automated research system attempted to edit its code to extend the time limits imposed on its experiments instead of shortening the runtime, made a system call that relaunched itself, and saved a checkpoint on every update until the files filled nearly a terabyte of storage (Lu et al., 2024). Monitoring a reasoning model's chain of thought caught reward hacking in agentic coding tasks, and training the model against that monitor produced reward hacking that the monitor no longer saw (Baker et al., 2025). Goodhart's law names the pattern: a measure that becomes a target stops measuring well, whether because selection on a noisy proxy selects the noise or because optimization pushes into regions where proxy and goal come apart (Manheim and Garrabrant, 2018).

No published report documents these behaviours in a recommender loop, but the recommender analogues are easy to list. A loop that can touch evaluation could drop hard cold-start rows from validation, write values derived from the label into the predictions file, change how the evaluator bins scores for calibration, or move the label threshold.

### C.2.3 When the Test Set Is Reachable

The third mechanism is test feedback. Two patterns are common in ordinary experiment code. One passes the test set as the early-stopping evaluation set of a gradient-boosting library, so that the number of trees is chosen on test data. The other points a hyperparameter search's objective at a test metric. Either turns the test set into a second validation set, and an agent asked to improve "the metric" will follow whichever path the code exposes. The telltale artifact is a results file that reports test metrics and no validation metric. Leakage between training and evaluation data is a recurring cause of irreproducible results across the sciences (Kapoor and Narayanan, 2023), and in recommender evaluation, splits that ignore the global timeline let future interactions leak into training and can change which model looks best (Ji et al., 2023). Chapter 4's point-in-time rules and Chapter 6's construction of aggregates from earlier windows are the leakage policy that a harness enforces.

The practical lesson is to audit existing scripts for these patterns before automating anything on top of them. The harness design on which this appendix's exercise is based, called the reference harness below, began with such an audit, and its contract records the outcome as a rule: test data may not be used for training, early stopping, model selection, preprocessing fits, search, or planning.

### C.2.4 Weak Baselines and Hidden Denominators

A gain is only as meaningful as its comparison. Many neural recommendation results could not be reproduced, and most of those that could were outperformed by simple, carefully tuned baselines (Ferrari Dacrema et al., 2019); well-tuned matrix factorization outperformed reported results on standard benchmarks (Rendle et al., 2019). Financial backtests face the same problem with trial counts, which are rarely reported even though the chance of a spurious winner grows with every strategy tried (Bailey et al., 2017). Even without deliberate fishing, analysis choices that depend on the data create a multiple-comparisons problem (Gelman and Loken, 2014). An agent's leaderboard without its count of failed and discarded trials is a backtest without its trial count.

### C.2.5 Writeups That Outrun Evidence

The last mechanism lives in the reports. An independent evaluation of one automated research system found that a large share of its experiments failed on coding errors, and that some of its generated papers contained hallucinated numerical results (Beel et al., 2025). Language models do not reliably

correct their own reasoning without external feedback (Huang et al., 2024), language-model evaluators recognize and favour their own generations (Panickssery et al., 2024), and research ideas generated by a language model lost more ground than human-written ideas once experts actually executed them (Si et al., 2026). An agent's account of its own results is a claim to check, never evidence by itself.

Table C.2 condenses these mechanisms into a reference, with the guard that later sections build for each and the risk that remains after the guard.

*Table C.2: Failure modes of automated search and their guards.*

| Failure mode | How it shows in artifacts | Structural guard | Residual risk |
|---|---|---|---|
| Test feedback into training or selection | Early-stopping or search objective names test; test metrics improve across trials | Test labels outside every agent workspace; evaluator refuses test without a promotion record; search objective must be a validation metric | Raw logs can reconstruct test labels unless the window is withheld |
| Adaptive overfitting of validation | Best-so-far rises in shrinking steps; near-duplicate configurations crowd the top | Append-only ledger; noise band; one-shot test; fresh validation window per research phase | Small true gains stay unresolvable on one window |
| Evaluator tampering | Diffs touch sealed files; hash mismatch; implausible metric jumps | Evaluator outside the writable area; hashes checked before every evaluation; metrics recomputed from predictions | Laptop sandboxes are weak |
| Silent subsetting | Row count below the manifest; shifted positive rate | Predictions joined on the sealed evaluation key set; missing keys fail the run | Low once key checks exist |
| Proxy and goal divergence | Primary metric up while calibration or slices worsen | Sealed guardrail metrics with floors; mandatory slice report | The sealed metric may itself be the wrong target |
| Feature leakage | One feature with near-perfect univariate AUC; anomalous train-to-validation gap | Banned names, per-feature audit, point-in-time rules, Critic review | Renaming evades name checks |

| Failure mode | How it shows in artifacts | Structural guard | Residual risk |
|---|---|---|---|
| Rediscovering dead ends | Duplicate ideas and repeated failures in the ledger | Proposer sees failures as well as successes; Critic checks the ledger | Semantic duplicates are hard to detect |
| Strawman baselines | Default-setting baselines; no simple baselines | Baselines sealed in the contract with equal tuning budgets | Budget parity is approximate |
| Protocol extended after results | Sweep bounds move toward the best value; robustness checks chosen late | Sweep bounds and robustness checks declared before running | Judgment calls remain |
| Constraint bypass and runaway | Runner settings changed; runs over budget; storage spikes | Limits enforced by the runner or operating system; quotas | Laptop sandboxes are weak |
| Unsupported numbers | A number in a writeup with no trial id | Every number cites a trial id; automated cross-check | Rounded numbers are harder to match |
| Hidden denominator | Leaderboard shorter than the count of run directories | Ledger records failures and timeouts; reports state the trial count | Low once enforced |
| Critic capture | Approvals that restate the proposal; no rejections | Clean-context Critic reading artifacts; deterministic checks first; Critic never the sole gate | Shared model priors |

## C.3 The Sealed Evaluation Harness

Most failures in Section C.2 share a root: the thing being optimized could reach the thing that judges it. The remedy is to decide what counts as better once, before the search begins, and to freeze that decision where no optimizing process can change it.

### C.3.1 What the Harness Is

An evaluation harness is a versioned evaluation contract together with the code that enforces it. The contract names a split manifest id, a short identifier such as `chapter6_yambda_pointwise_v1` that stands for one frozen choice of data, label, split, evaluator, and rules. "Sealed" means that no role that optimizes the metric may change any of these, and that any change, however small, produces a new manifest id. Section 15.9.3 versions every served artifact so that it can be named and retrieved, and

Section 15.9.5 asks each candidate's record to name both its model-registry version and the version of the evaluation code and data that judged it. The manifest id is that second version. The two are different objects: one candidate can be judged under several harness versions, and one harness version judges many candidates.

## C.3.2 Sealed Artifacts

The reference harness seals seven things. The dataset snapshot and its source files fix the population. A timestamp policy fixes units and granularity, so that split boundaries are exact. The label definition fixes the target, here a play of at least half a track. The split manifest fixes the training window, a 30-minute gap, a one-day validation window, another 30-minute gap, and a one-day test window, following the temporal split design of Chapter 3 and the gap of the Chapter 6 tutorial. The evaluator and its metric set fix what "better" means. The leakage checks fix which inputs are allowed. The promotion rules fix what earns a test evaluation.

Five further items belong in the seal for any loop that runs at volume. Sealed baselines, tuned with the same budget as the search, make "improvement" relative to something fixed. Guardrail metrics with floors, such as log loss, calibration error, and performance on cold-start slices, bound Goodhart effects. An evaluation key set, the list of row identifiers that every validation submission must cover, prevents silent subsetting. A declared test budget bounds how many times the test window may be read. Content hashes of the sealed files make tampering evident. Protocol details such as candidate sampling belong here too, because evaluating on sampled candidates instead of the full catalogue can change which model ranks first (Krichene and Rendle, 2020).

Figure C.3 shows the boundary and the only three things that may cross it.

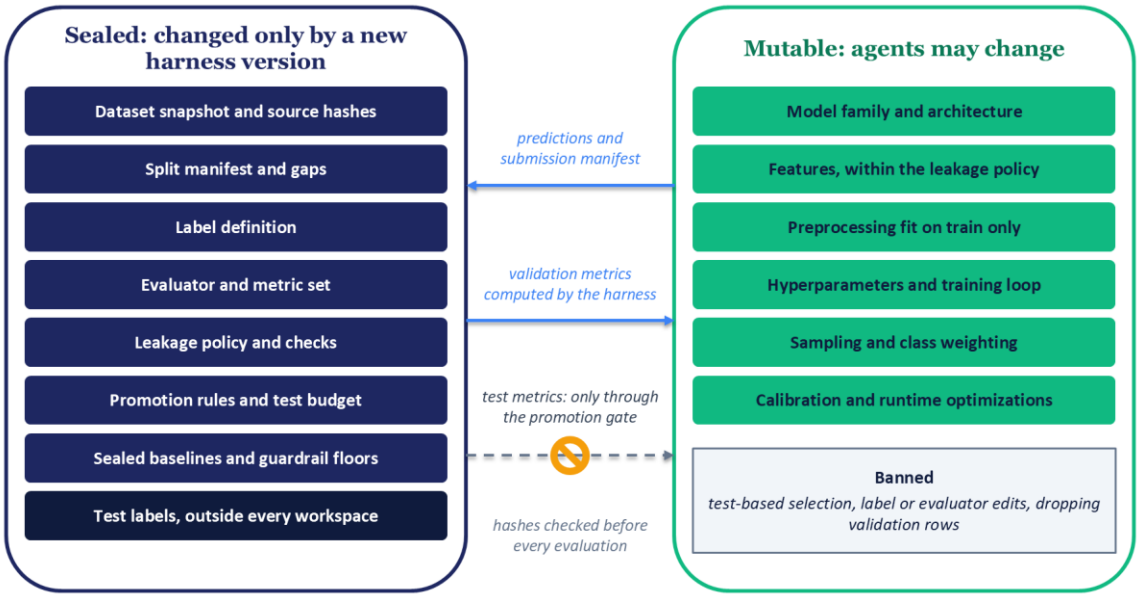



*Figure C.3: The sealed harness and the mutable research surface. Agents may change anything in the right panel and nothing in the left. Predictions and a submission manifest cross from the research side, harness-computed validation metrics cross back, and test metrics leave only through the promotion gate, with hashes of the sealed files checked before every evaluation.*

### C.3.3 The Mutable Research Surface and Banned Behaviour

Inside the boundary, agents may change the model family and architecture, features within the leakage policy, preprocessing that is fit on training data only, hyperparameters, the training loop, regularization, sampling and class weighting, calibration, and runtime optimizations, and they may run any analysis or ablation on validation data. "Within the leakage policy" carries real content for the Chapter 6 ranker: aggregates such as a user's completion rate come from the same column as the label, so the statistics applied to a row must come only from events before that row's window, as Chapter 6 constructs them and Chapter 4's point-in-time rules require.

Some behaviour is banned outright. Test data may never drive training, early stopping, checkpoint choice, model selection, or search. The label, split, evaluator, and leakage checks may not be changed. Preprocessing may not be fit on validation or test rows, validation rows may not be dropped or reweighted at evaluation time, and no result counts without predictions that the harness itself scored. For the Chapter 6 task the contract also bans the raw timestamp and the played-ratio column as features, since one leaks position in time and the other is the label's source. Three operational bans complete the list: agents may not edit the runner's limits, read the sealed directory, or write to another agent's workspace. Table C.3 gathers the contract in one place.

*Table C.3: A harness contract for the Chapter 6 ranker.*

| Item | Status | Who may change it | How it is verified |
|---|---|---|---|
| Dataset snapshot and source files | Sealed | Auditor, by a new version | Size, modification time, and content hash checked before each run |
| Timestamp policy and label definition | Sealed | Auditor, by a new version | Harness computes labels; agents never write them |
| Split manifest with gaps | Sealed | Auditor, by a new version | Boundaries, gaps, and durations validated exactly |
| Evaluator and metric set | Sealed | Auditor, by a new version | Hash check; metrics recomputed from predictions |
| Leakage policy and checks | Sealed | Auditor, by a new version | Run on every submission |
| Promotion rules and test budget | Sealed | Auditor, by a new version | Gate in code; append-only test ledger |
| Sealed baselines and guardrail floors | Sealed | Auditor, by a new version | Baselines rerun at harness creation; floors checked at the gate |
| Evaluation key set | Sealed | Auditor, by a new version | Coverage check on every evaluation |

| Item | Status | Who may change it | How it is verified |
|---|---|---|---|
| Test labels | Sealed | Nobody inside the loop | Stored outside every agent workspace |
| Model code, features, hyperparameters | Mutable | Implementation agents, in their own workspace | Leakage checks; Critic review |
| Preprocessing fit on training data | Mutable | Implementation agents | Declared fit split in the submission manifest |
| Proposer strategy and research notes | Mutable | Research agent | Critic review against the ledger |
| Test-based selection or early stopping | Banned | Nobody | Metadata checks; search objective must be a validation metric |
| Dropping or reweighting validation rows | Banned | Nobody | Key-set coverage check |
| Editing runner limits or sealed files | Banned | Nobody | Limits set outside the agent; hash check |

### C.3.4 How Strongly the Seal Holds

A seal is only as strong as its enforcement, and enforcement comes in four levels. At the lowest, the seal is an instruction: the agent's prompt says not to touch the test set. Next, the seal is tamper-evident: hashes of sealed files and an append-only ledger reveal a change after it happens. Next, it is access-controlled: agent processes run under permissions or in containers that cannot read test labels or write sealed files. At the highest level, the harness is an evaluation service, a separate process that receives predictions and returns only aggregate metrics, which can be coarsened in the manner of Blum and Hardt (2015) by reporting a new score only when it beats the best so far by a margin. The finding that reward hacking rises when the scoring function is visible (Von Arx et al., 2025) argues for the upper two levels whenever the loop runs unattended.

Published systems place enforcement in different places. One recommender system expresses safety thresholds as instructions in its online agent's prompt and validates candidates in live experiments against delayed north-star metrics, so that its final check lives online (Wang et al., 2026). A sealed offline harness places a check earlier, before any traffic is spent. The reference harness is candid about its own level: it seals the test window with a permission flag that the calling code sets, and it checks leakage by feature name, which places it at the tamper-evident level at best. The laptop exercise in Section C.8 sits at the same level, which is why its second part asks you to try to break it.

### C.3.5 Who May Change the Seal

Only one role may change the seal: the Harness Auditor, and only by creating a new harness version with a new manifest id. That authority must sit outside every role that optimizes the metric, and in the

design this appendix recommends a human owner signs off each new version. Leaderboards and noise bands never merge across harness versions, because a score under one version and a score under another answer different questions.

Five events justify a new version: an evaluator bug, newly discovered leakage, an exhausted test budget, a data refresh, and a change of task or label. Each new version follows the same steps. The Auditor first audits the code that will be trusted, then freezes new boundaries and gaps, records source metadata, hashes, the evaluation key set, and a test budget, reruns the sealed baselines, and writes the reason for the change in a harness changelog.

## C.4 Roles, Context, and Handoffs

With the harness fixed, the next question is how to organize the agents that work inside it. The answer starts small and adds roles only where they buy a check that would otherwise be missing.

### C.4.1 Start with One Loop

The simplest design that works is the right starting point, and for some problems that means no agentic system at all (Schluntz and Zhang, 2024). One published ads-ranking system uses a single agent that walks five stages, from hypothesis generation through result analysis and a shared knowledge store, against a sandboxed copy of the codebase, with a human checkpoint at every stage boundary where an engineer can approve, modify, or stop the session (Gao et al., 2026). A practitioner account of multi-agent systems reports that the designs that work keep writes single-threaded and give reviewing agents a clean context (Yan, 2026). A single proposing agent, a deterministic sealed evaluator, and a human checkpoint therefore make a complete design. The two-level model below adds roles when the volume of trials makes one reviewer too few; no published comparison shows that it outperforms a single agent with checkpoints, so treat the choice as a design decision.

### C.4.2 The Two-Level Operating Model

At Level 0 sits the Harness Auditor. It audits existing code for test feedback and leakage, builds the seal, writes the research brief and the instructions for each role, and runs the single sealed-test evaluation. At Level 1 sits a research cell. A Research Scientist plans trials and writes each proposal into a research log. A Critic reviews proposals before they run and results after, against a fixed checklist. Two implementation agents, MLE Agent A and MLE Agent B, implement and run trials, each in its own workspace, and submit predictions to the sealed evaluator.

The roles run in separate contexts for two reasons. A model's context window is the text it can attend to in one call, and models use information placed in the middle of a long context less reliably than information at its start or end (Liu et al., 2024), so a single agent carrying the whole history of a long search degrades. Separate context windows also let each role see only what its job requires, although vague instructions to subagents lead them to duplicate each other's work (Hadfield et al., 2025). Figure C.4 shows the model and the written artifacts that carry each handoff.

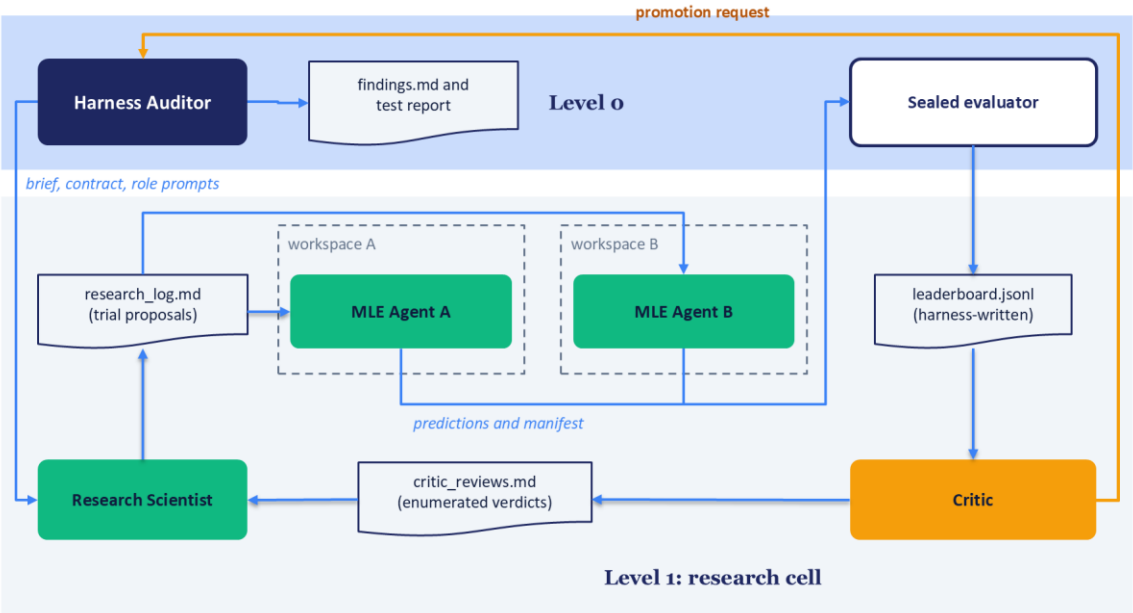



*Figure C.4: The two-level operating model. The Harness Auditor at Level 0 writes the brief and contract, owns the sealed evaluator, and alone runs the sealed-test evaluation. In the Level 1 research cell, the Research Scientist proposes trials through the research log, two MLE agents work in disjoint workspaces and submit predictions to the sealed evaluator, and the Critic reviews the leaderboard and requests promotion.*

### C.4.3 Handoffs Through Written Artifacts

Roles hand work to each other through files, never through shared conversation. A research brief states the question and scope. The contract states the seal. The research log holds trial proposals, each with its rationale, sweep bounds, and planned robustness checks. The Critic's reviews file holds its verdicts. The leaderboard, written only by the harness, holds one structured record per trial. A findings file holds claims, each tied to the trials that support it. Each trial also has its own run directory containing the configuration, a submission manifest that declares the features and their fit split, the validation predictions, the harness-computed validation metrics, and the agent's notes.

Industrial systems arrived at the same idea by different routes. One keeps an experiment journal of every configuration with its offline and online results, which every agent reads (Wang et al., 2026). Another keeps a shared knowledge store of per-technique and per-model outcomes as text files versioned in source control (Gao et al., 2026). A third turns negative online results into a failure memory that prunes future ideas (AgentX Team, 2026). That convergence suggests that the written record carries the system, a theme Section C.6 develops.

### C.4.4 The Critic: Useful and Limited

The Critic works from a checklist (test use, leakage, budget parity with baselines, sweep bounds and robustness checks declared in advance, every value tried logged) and returns one of five verdicts: approve for run, revise before run, reject, approve for promotion, or reject promotion. Its limits are well documented. Verification failures form one of the three broad categories in a taxonomy of multi-agent system failures (Cemri et al., 2025), which suggests that adding a reviewing role does not by itself close the gap. A Critic built on the same model as the proposer may also favour text that resembles its own

(Panickssery et al., 2024), and self-correction without external feedback is unreliable (Huang et al., 2024).

The design responds in three ways. Deterministic checks (hashes, coverage, leakage, schema) run before the Critic sees anything. The Critic reads artifacts and records instead of the proposing agent's reasoning. And the Critic is never the only gate. Using a different model family for the Critic is an option, though it reduces shared bias without removing it.

### C.4.5 Independence of Parallel Workers

The two MLE agents never share a workspace or memory until review. That keeps one agent's bug from contaminating the other's results, but it does not make them independent witnesses. Agents built on the same base model share its priors, so when both arrive at the same idea with the same rationale, the agreement is weak replication. Stronger replication changes something that matters: the seed, the data slice, or the method.

### C.4.6 Published Industrial Designs

Table C.4 describes four systems published in 2026 for agent-driven development of production ranking and recommendation models. All four reports are authored by the companies that built the systems, and any result they contain is that source's reported result under its own conditions, so the table describes designs and quotes no results. One system estimates the compute cost of a plan and confirms it with an engineer before execution (Kumar et al., 2026). Another admits edits to its own agent scaffold only after a paired replay shows the edited version performing better on identical tasks (AgentX Team, 2026). A third ends each session in "either a documented proposal or a documented null result" (Gao et al., 2026).

*Table C.4: Four published designs for agent-driven recommender model development.*

| System and organization | Topology | Where evaluation is grounded | Human role | Outcome memory |
|---|---|---|---|---|
| Ranking Engineer Agent (REA), Meta (Kumar et al., 2026) | Hypothesis generation, planning, and an executor that hibernates while training jobs run | Results of its own training jobs | Oversight at strategic decision points; engineers approve compute budgets | Historical insights database of outcomes, metrics, and configurations |
| A-MLE, Meta (Gao et al., 2026) | One agent, five stages, sandboxed codebase copy | Offline evaluation suite, with significance against a rolling baseline | Checkpoint at every stage boundary | Shared knowledge store versioned in source control |
| Self-Evolving Recommendation System, Google and YouTube | Offline agent in a fast loop; online | Proxy metrics offline; delayed north-star metrics | Supplies the initial research idea; reviews | Experiment journal of all |

| System and organization | Topology | Where evaluation is grounded | Human role | Outcome memory |
|---|---|---|---|---|
| (Wang et al., 2026) | agent in a slow loop | in live experiments | experiment metrics | configurations and results |
| AgentX, Kuaishou (AgentX Team, 2026) | Four stages in a closed loop, from brainstorming to online evaluation and scaffold evolution | Online A/B results, with a guardrail veto | Admission gate before ideas are implemented | Failure memory built from negative results |

## C.5 Validation-First Optimization and the Promotion Gate

The seal says what may not change. The promotion gate turns it into a procedure: an ordered set of checks that a candidate must pass before anyone spends the test window on it.

### C.5.1 Validation First, Test Once

Agents optimize only against validation data, and evaluating on the test window is an action reserved to the Harness Auditor. The reason is the one Section C.2.1 gave: each query to a holdout leaks information about it, and repeated adaptive queries void the guarantees that make a holdout trustworthy (Dwork et al., 2015). Coarse feedback limits how much each query leaks (Blum and Hardt, 2015), and the strictest coarsening is to give the loop no test feedback at all.

The contract therefore declares a test budget in advance: the number of sealed-test evaluations this harness version allows, and for which candidates. The Chapter 6 exercise sets it to three, spent on the winner of each of two search arms and on a baseline configuration fixed before any trial ran. Once the budget is spent, further testing needs a new harness version with a fresh test window.

### C.5.2 The Gate

Figure C.5 lays out the gate as seven checks in a fixed order, followed by the single test evaluation.

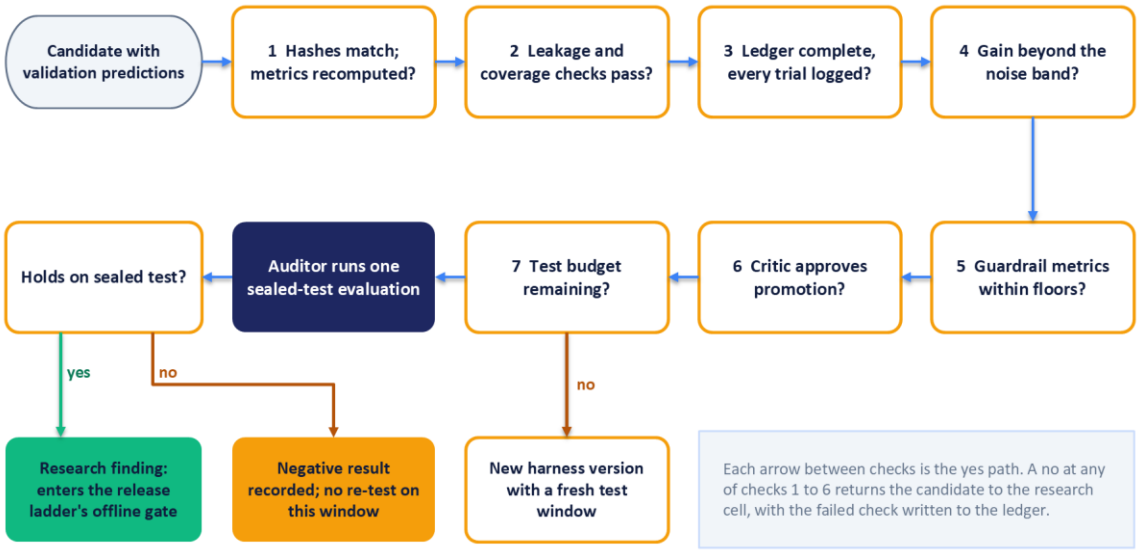



*Figure C.5: The promotion gate as an ordered procedure. A candidate passes seven checks, from recomputed metrics to remaining test budget, before the Harness Auditor runs one sealed-test evaluation. The outcome is a research finding for Chapter 15's offline validation gate, a recorded negative result, or, once the budget is spent, a new harness version with a fresh test window.*

The first check confirms that the sealed files' hashes match and recomputes the candidate's metrics from its predictions; the harness never accepts metrics reported by an agent. The second runs the leakage checks and a coverage check that joins the predictions to the sealed evaluation key set, failing on any missing or duplicated key. The third confirms that the ledger is complete, so that the promotion request carries its denominator: how many trials were attempted, how many failed or timed out, and how many configurations were compared.

The fourth check asks whether the gain exceeds the noise band. The noise band is the spread of the sealed baseline's validation metric across several training seeds, widened by a bootstrap over users, which resamples users with replacement and recomputes the metric on each resample. Seeds alone understate the noise, because data sampling, initialization, and hyperparameter choices all contribute to the variance of a benchmark result (Bouthillier et al., 2021), and the bootstrap captures the sampling part. The promotion threshold is set relative to this band in the contract before the search begins.

The fifth check confirms that guardrail metrics stay within their floors. A guardrail metric is one that must not degrade beyond a set limit even when the primary metric improves; here, log loss, calibration error, and AUC on cold-start slices. The sixth requires the Critic's promotion verdict. The seventh confirms that test budget remains. A "no" at any of the first six checks returns the candidate to the research cell with the failed check written to the ledger, and a "no" at the seventh triggers a new harness version.

One published system takes a different approach to its comparison point, computing significance against a rolling baseline, the current reference configuration, instead of a frozen snapshot, and rerunning automatically when variance within a run exceeds a threshold (Gao et al., 2026). A rolling baseline tracks a production model that keeps changing, which suits a team's day-to-day decisions. A sealed baseline keeps every comparison within one harness version on the same footing, which is what a claim about selection optimism needs. The two answer different questions, and a team can keep both.

### C.5.3 Reading the Test Result

What "holds" means is written in the contract before any trial runs, for example that the candidate's test score exceeds the sealed baseline's test score by more than the noise band. If the result holds, the candidate becomes a research finding and goes to the offline validation gate of Section 15.9.5. Research promotion and release promotion are separate decisions: the first says the gain survived one sealed-test evaluation, and the second says the model is fit to enter the release ladder of shadow and canary stages. Online experiments then decide whether the change is better for users, and a loop that sends many candidates online meets the many-variant problem of Section 16.4.6.

If the result does not hold, the outcome is a negative result, recorded with the same care as a positive one. Re-tuning and re-testing on the same window would turn the test set into a validation set, so it is banned; a fresh window means a new harness version. Published industrial designs treat null and negative results as first-class outputs, whether as documented null results (Gao et al., 2026) or as failure memory that steers later search (AgentX Team, 2026).

### C.5.4 Budget-Aware Comparison

Comparing a search's best result with a baseline's single run flatters the search. Expected maximum validation performance, reported as a function of the number of configurations tried, makes the comparison fair by showing what each method is expected to reach with a given number of trials (Dodge et al., 2019). Random search is the reference for that comparison. It is simple, more efficient than grid search when only a few hyperparameters matter (Bergstra and Bengio, 2012), and competitive enough in architecture search that more elaborate methods should be compared against it at equal budget (Li and Talwalkar, 2019; Lindauer and Hutter, 2020).

Language-model proposers are a recent entrant. One benchmark study reports that, within small budgets, language models proposing hyperparameters matched or outperformed Bayesian optimization; that is the authors' reported result under their conditions (Zhang et al., 2023). Whether an agent proposer beats random search on a given recommender problem is an empirical question, and the harness is the instrument that answers it: equal budgets, the same seal, the same gate, and a ledger for each arm.

## C.6 The Record Is the Product

A loop that runs hundreds of trials produces a handful of candidates worth promoting and a great deal of evidence about what did not work. The record of all of it is the loop's real output, because it is what lets anyone, later, decide how much a claimed improvement deserves to be believed.

### C.6.1 The Trial Record

Every trial, including those that crash or time out, gets one structured record written by the harness. The record holds a trial id, the agent that ran it, the idea in a sentence, and a status drawn from a fixed set: completed, failed, or timed out. It holds the full configuration, the validation metrics the harness computed, the training metrics, and the Critic's verdict. It names the split manifest id and a code manifest, which lists a hash for each source file the trial used. It records the hardware, the runtime, the random seeds, the dataset snapshot, a timestamp, and free-text notes.

Three properties make the record trustworthy. The ledger is append-only, so a record can be corrected only by adding a new one that refers to it. The harness writes it, never the agent, so a metric in the

ledger is always a metric the harness computed. And every report states its denominator, the number of trials behind it, so a reader can judge how much selection went into the best result. Section 15.9.3 versions the training data and its window, the feature definitions, and the model weights so that any served artifact can be reproduced. The trial record adds the harness version and the search history, which lets a promoted candidate carry into the registry both what it is and how it was chosen.

### C.6.2 Levels of Evidence

Evidence for an agent-reported result comes in five levels, each cheaper to fake than the one above it. At the lowest level is a claim in a writeup, a sentence that says a change helped. Above it is a ledger row, which at least ties the claim to a trial id and a configuration. Above that are the run artifacts: the predictions, the submission manifest, and the notes that let someone inspect what was actually done. Next are metrics recomputed by the harness from those predictions, which remove any doubt about the arithmetic. At the top is an independent rerun from a clean checkout, which shows that the predictions themselves came from the stated code and data.

Automated research systems make the lowest level abundant: an independent evaluation found hallucinated numbers in the papers one such system wrote (Beel et al., 2025). The working rule follows directly. Every number in a findings file cites the trial id it came from, a script checks each cited number against the ledger, and any claim that will leave the team, such as a promotion request, needs evidence from the top two levels.

### C.6.3 State from Records, Not Prose

The reference harness's own records illustrate why state must come from records. In one run, the research log described a candidate as the current promoted candidate while the findings file stated that nothing had yet been promoted. In the same run, the Critic's reviews used verdict labels that did not appear in its defined vocabulary of five. Neither failure involved a wrong metric; both made it impossible to answer, from the files alone, the question of what state the research was in.

The fix is to derive state from schema-validated records instead of prose. Verdicts are enumerated values, and a write with an unknown verdict fails validation. Promotion status is a field computed from gate records, never a phrase in a log. Writeups are generated from the records, or checked against them, so that the narrative cannot drift away from what the ledger says. Appendix D, Benchmarking Structured-Output Scorers, takes up a related measurement problem for language-model scorers whose typed outputs feed a recommender.

## C.7 Cost, Review Capacity, and When Not to Build the Loop

An agentic loop has three costs: the compute that trains and evaluates each trial, the language-model calls that drive the agents, and the human time that reviews what they produce. The first two are easy to meter. The third is easy to forget, and it decides whether the loop produces knowledge or only volume.

### C.7.1 Where the Cost Goes

Compute grows with trials, and a loop that makes trials cheap to propose will propose many. Model calls grow faster than intuition suggests: one published account of a multi-agent research system reports that it consumes many times the tokens of an ordinary chat interaction, because each subagent reads and reasons over its own context (Hadfield et al., 2025). Token prices and model capabilities change quickly,

so any cost estimate belongs in a dated planning document and should be refreshed before each budget decision.

Industrial designs make cost a checkpoint instead of an afterthought. One estimates the compute cost of an experiment plan and has an engineer confirm the approach before execution (Kumar et al., 2026). A team building its own loop can do the same by writing a compute budget into the contract and requiring sign-off when a plan would exceed it.

### C.7.2 Review Capacity Binds

Every promotion request, every dispute between the Critic and the Research Scientist, and every evidence package lands on a human reviewer. When a loop produces more of these than its reviewers can examine, approval turns into rubber-stamping, and the gate in Section C.5 stops being a gate. Three controls keep the load inside capacity: a cap on promotion requests per review cycle, a standard evidence package so that every request can be read in the same way, and human checkpoints at stage boundaries, which one published system uses to stop a bad code change before a training job launches and a miscalibrated evaluation before a proposal is written (Gao et al., 2026). The useful throughput of the loop is the number of candidates a team can review properly, whatever the agents can produce.

### C.7.3 Stopping Rules

A loop needs four limits, each enforced by the runner outside the agent's reach: a total wall-clock budget, a timeout per trial, a maximum number of trials, and a patience rule that stops the search when a set number of consecutive trials fails to improve the best validation result. Section C.2.2 showed why the runner must hold them: a system that can edit its own limits may extend them (Lu et al., 2024). The stopping budgets are declared before the search starts. Stopping because the results look good is a form of peeking, the same problem Section 16.4.5 describes for online experiments: it selects the moment when noise favours the candidate.

### C.7.4 When Not to Build It

The case for an agentic loop is strongest where a machine-gradeable evaluator already exists and is trusted. One evaluator-driven coding agent reports improved algorithms in mathematics and in computing infrastructure found in exactly that setting, with automated evaluators scoring every candidate program (Novikov et al., 2025), and a practitioner account notes that notable public multi-agent demonstrations share a simple, verifiable success criterion (Yan, 2026).

The loop is the wrong tool in five situations. If the evaluator is unsound, with leakage, a weak label, or a metric that does not track the goal, a loop will optimize the flaw. If validation data is small, every query buys more overfitting than signal. If each trial is expensive, the loop's advantage in volume disappears. If review capacity is thin, its output cannot be checked. And if the open questions concern labels or objectives, they are questions about the seal itself, which belong to the human owner and a new harness version. In each of these cases, a scripted sweep or random search under the same harness gives the same discipline at a fraction of the cost, and the simplest design that works is the right one (Schluntz and Zhang, 2024).

The central claim of this appendix also has a falsifier. If, across several harness versions, the sealed-test gap of searched winners matches that of a fixed baseline, the search space is too small or the validation window too large for selection to overfit, and the elaborate guards of Sections C.3 to C.6 are buying little on that problem.

## C.8 Exercises: A Guarded Search Loop on the Chapter 6 Ranker

The companion code for these exercises lives in `code/appendix_agentic_autoresearch/`. It needs no language-model calls and no API key, and it runs on a CPU. Deterministic proposers play the agent's role: a function `propose(history)` that receives the ledger so far and returns the next configuration. An optional replay proposer reads `proposers/proposals_replay.jsonl`, a short hand-written file labelled as not model output.

### Exercise C.1: A Guarded Search Loop

The data is the Yambda snapshot used in Section 6.8. The frozen split uses a training lookback of 7 days, a history window of the same 7 days before the training window, gaps of 1,800 seconds (30 minutes), and validation and test windows of one day each. Each trial imports the Chapter 6 feature pipeline and trains XGBoost with early stopping on the last tenth of the training window. A full run budgets three trials per arm. The `--synthetic` flag uses two trials per arm and turns the replay arm on. Replay stays off on the Yambda path unless `--replay` is passed, and replay is not one of the three test candidates.

Work through four steps. First, run `build_harness.py`. It builds the validation features and labels, writes the test labels to a `sealed/` directory, and records file hashes and a test budget of three in the seal record when the harness is built, so a later edit to `contract.yaml` fails the hash check. The validation labels carry `row_id`. Before any trial, `contract.yaml` fixes the lookback, the per-arm trial budget, the seeds, the baseline configuration, the noise-band method (five baseline seeds plus a bootstrap over users), and the promotion threshold. Second, run `loop.py` for two arms with equal budgets: random search over `search_space.yaml`, and a greedy arm that perturbs the current best configuration and keeps or discards the change. The loop never opens `sealed/` and logs every trial. Third, run `promote_and_test.py` once for each of `baseline`, `random`, and `greedy`. Each run spends one test evaluation when the hashes match, the checks pass, and budget remains. A failed check is logged and does not spend the budget. Fourth, run `report.py`.

The report plots best-so-far validation AUC, the expected-maximum curve, the noise band, and the denominator. The optimism gap is validation AUC minus test AUC. Shift-adjusted optimism subtracts the baseline gap.

Read the result in one of four ways. Shift-adjusted optimism above the noise band means the search overfit the validation day. An in-band result, a greedy arm no better than random search at equal budget, or a rejected gate is a valid null. Roelofs et al. (2019) find the hyperparameter space of gradient-boosted trees small and smooth relative to a day of validation data, so query volume buys little overfitting. The shipped search space is hyperparameters only. It does not let the proposer add or drop features. `simulate_best_of_n.py` reproduces Figure C.2: seed 20261109, 5,000 repetitions, candidate counts 1, 2, 4, 8, 16, 32, 64, 128, and 256, quality drawn from a normal distribution with standard deviation 0.3, independent unit noise on validation and on test, and the dashed average-candidate series. The README records about 22 seconds for the six-command synthetic sequence on CPU. The Yambda run was not timed.

### Exercise C.2: Red-Team the Seal

The second exercise builds four small dishonest submissions and runs the checks in `harness/leakage.py`. `loop.py` and `promote_and_test.py` call the same checks.

The first submission uses a column named `timestamp`, `played_ratio_pct`, or `label`, or any feature name containing `label`. `check_feature_columns` rejects each of those names. It also rejects the

feature names `target`, `split`, `segment`, and `row_id`. It accepts `hour_of_day` and `day_of_week`, which the Chapter 6 pipeline emits from the timestamp.

The second submission drops validation rows. `check_coverage` catches a prediction file that is missing `row_id` values from the validation labels.

The third submission writes the validation labels as scores. `check_reproduction` compares the metrics stored on the research record with metrics recomputed from the prediction file and the validation labels. The label scores fail that comparison when the record still holds the saved model's metrics. The saved model is not reloaded for this check. `predict_with_saved_pipeline` scores the sealed test rows only after the gate has passed.

The fourth submission is a renamed timestamp together with a renamed label feature, with neither name banned and neither containing `label`. A renamed timestamp and a renamed label feature can pass every shipped check. The seal matters more than detecting misuse after the fact.

## C.9 Summary

Coding agents can run the mechanical half of recommender research: editing a training script, launching it, reading the metric, and trying again. That capacity amplifies whatever experimental discipline surrounds it. A loop that reuses one validation split hundreds of times, sees or edits its scorer, or reaches test data will produce convincing improvements that do not survive fresh data, and it will produce them faster, and in greater volume, than a human team working the same loop.

The trustworthy unit of automation is the sealed evaluation harness: a versioned contract that freezes the data snapshot, label, split, evaluator, leakage policy, baselines, guardrails, and promotion rules, enforced by code that no optimizing role can change. Agents earn autonomy inside that boundary. Three things stay with an owner outside the loop: the boundary itself, the single evaluation on sealed test data, and the decision to promote.

Roles hand work to each other through written artifacts, and a Critic helps only when deterministic checks run first and it is never the sole gate. The promotion gate recomputes metrics, checks leakage and coverage, requires a gain beyond a noise band built from seeds and a user-level bootstrap, enforces guardrail floors, and spends test data within a budget declared in advance. Its output is a research finding or a recorded negative result, and a finding still faces Chapter 15's release ladder and Chapter 16's online experiments.

The system's product is an auditable record of everything tried, failures included, from which a few claims can be defended. When the evaluator is weak, validation data is small, trials are expensive, or review capacity is thin, a scripted sweep under the same harness is the better tool. The claim has a falsifier: if searched winners show no more sealed-test optimism than a fixed baseline across harness versions, the guards are buying little on that problem.

## References

AgentX Team (2026). "AgentX: Towards Agent-Driven Self-Iteration of Industrial Recommender Systems." arXiv preprint arXiv:2606.26859. https://arxiv.org/abs/2606.26859

Bailey, D. H., Borwein, J. M., López de Prado, M., and Zhu, Q. J. (2017). "The Probability of Backtest Overfitting." Journal of Computational Finance 20(4). Infopro Digital. https://doi.org/10.21314/JCF.2016.322

Baker, B., Huizinga, J., Gao, L., Dou, Z., Guan, M. Y., Madry, A., Zaremba, W., Pachocki, J., and Farhi, D. (2025). "Monitoring Reasoning Models for Misbehavior and the Risks of Promoting Obfuscation." arXiv preprint arXiv:2503.11926. https://arxiv.org/abs/2503.11926

Beel, J., Kan, M.-Y., and Baumgart, M. (2025). "Evaluating Sakana's AI Scientist: Bold Claims, Mixed Results, and a Promising Future?" SIGIR Forum 59(1). ACM. https://doi.org/10.1145/3769733.3769747

Bergstra, J., and Bengio, Y. (2012). "Random Search for Hyper-Parameter Optimization." Journal of Machine Learning Research 13. JMLR. https://jmlr.org/papers/v13/bergstra12a.html

Blum, A., and Hardt, M. (2015). "The Ladder: A Reliable Leaderboard for Machine Learning Competitions." Proceedings of the 32nd International Conference on Machine Learning, PMLR 37. PMLR. https://proceedings.mlr.press/v37/blum15.html

Bouthillier, X., Delaunay, P., Bronzi, M., Trofimov, A., Nichyporuk, B., Szeto, J., Sepah, N., Raff, E., Madan, K., Voleti, V., et al. (2021). "Accounting for Variance in Machine Learning Benchmarks." Proceedings of Machine Learning and Systems 3 (MLSys 2021). https://arxiv.org/abs/2103.03098

Cawley, G. C., and Talbot, N. L. C. (2010). "On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation." Journal of Machine Learning Research 11. JMLR. https://jmlr.org/papers/v11/cawley10a.html

Cemri, M., Pan, M. Z., Yang, S., Agrawal, L. A., Chopra, B., Tiwari, R., Keutzer, K., Parameswaran, A., Klein, D., Ramchandran, K., et al. (2025). "Why Do Multi-Agent LLM Systems Fail?" arXiv preprint arXiv:2503.13657. https://arxiv.org/abs/2503.13657

Dodge, J., Gururangan, S., Card, D., Schwartz, R., and Smith, N. A. (2019). "Show Your Work: Improved Reporting of Experimental Results." Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing. Association for Computational Linguistics. https://aclanthology.org/D19-1224/

Dwork, C., Feldman, V., Hardt, M., Pitassi, T., Reingold, O., and Roth, A. (2015). "The Reusable Holdout: Preserving Validity in Adaptive Data Analysis." Science 349(6248). AAAS. https://doi.org/10.1126/science.aaa9375

Ferrari Dacrema, M., Cremonesi, P., and Jannach, D. (2019). "Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches." Proceedings of the 13th ACM Conference on Recommender Systems. ACM. https://doi.org/10.1145/3298689.3347058

Gao, E., Sunkara, V. K., Guan, J., Jia, Q., Xu, H., Ji, X., Wong, S., Chavali, S. T., Vaishnavi, P., Pandhi, A., et al. (2026). "Agentic ML Exploration (A-MLE) for Ads Ranking." arXiv preprint arXiv:2609.08248. https://arxiv.org/abs/2609.08248

Gelman, A., and Loken, E. (2014). "The Statistical Crisis in Science." American Scientist 102(6). Sigma Xi. https://doi.org/10.1511/2014.111.460

Hadfield, J., Zhang, B., Lien, K., Scholz, F., Fox, J., and Ford, D. (2025). "How We Built Our Multi-Agent Research System." Anthropic Engineering. https://www.anthropic.com/engineering/multi-agent-research-system

Huang, J., Chen, X., Mishra, S., Zheng, H. S., Yu, A. W., Song, X., and Zhou, D. (2024). "Large Language Models Cannot Self-Correct Reasoning Yet." International Conference on Learning Representations (ICLR 2024). https://arxiv.org/abs/2310.01798

Ji, Y., Sun, A., Zhang, J., and Li, C. (2023). "A Critical Study on Data Leakage in Recommender System Offline Evaluation." ACM Transactions on Information Systems 41(3). ACM. https://doi.org/10.1145/3569930

Kapoor, S., and Narayanan, A. (2023). "Leakage and the Reproducibility Crisis in Machine-Learning-Based Science." Patterns 4(9). Cell Press. https://doi.org/10.1016/j.patter.2023.100804

Karpathy, A. (2026). "autoresearch." GitHub repository. https://github.com/karpathy/autoresearch

Krichene, W., and Rendle, S. (2020). "On Sampled Metrics for Item Recommendation." Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. ACM. https://doi.org/10.1145/3394486.3403226

Kumar, A., Gao, E., Levi, M., Yadawad, S., Wong, S., Iyer, S., and Sunkara, V. K. (2026). "Ranking Engineer Agent (REA): The Autonomous AI Agent Accelerating Meta's Ads Ranking Innovation." Engineering at Meta. https://engineering.fb.com/2026/03/17/developer-tools/ranking-engineer-agent-rea-autonomous-ai-system-accelerating-meta-ads-ranking-innovation/

Li, L., and Talwalkar, A. (2019). "Random Search and Reproducibility for Neural Architecture Search." Proceedings of the 35th Conference on Uncertainty in Artificial Intelligence, PMLR 115. PMLR. https://arxiv.org/abs/1902.07638

Lindauer, M., and Hutter, F. (2020). "Best Practices for Scientific Research on Neural Architecture Search." Journal of Machine Learning Research 21(243). JMLR. https://jmlr.org/papers/v21/20-056.html

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. (2024). "Lost in the Middle: How Language Models Use Long Contexts." Transactions of the Association for Computational Linguistics 12. MIT Press. https://doi.org/10.1162/tacl_a_00638

Lu, C., Lu, C., Lange, R. T., Foerster, J., Clune, J., and Ha, D. (2024). "The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery." arXiv preprint arXiv:2408.06292. https://arxiv.org/abs/2408.06292

Manheim, D., and Garrabrant, S. (2018). "Categorizing Variants of Goodhart's Law." arXiv preprint arXiv:1803.04585. https://arxiv.org/abs/1803.04585

Ng, A. Y. (1997). "Preventing 'Overfitting' of Cross-Validation Data." Proceedings of the Fourteenth International Conference on Machine Learning. Morgan Kaufmann. https://mlanthology.org/icml/1997/ng1997icml-preventing/

Novikov, A., Vũ, N., Eisenberger, M., Dupont, E., Huang, P.-S., Wagner, A. Z., Shirobokov, S., Kozlovskii, B., Ruiz, F. J. R., Mehrabian, A., et al. (2025). "AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery." arXiv preprint arXiv:2506.13131. https://arxiv.org/abs/2506.13131

Panickssery, A., Bowman, S. R., and Feng, S. (2024). "LLM Evaluators Recognize and Favor Their Own Generations." Advances in Neural Information Processing Systems 37 (NeurIPS 2024). https://arxiv.org/abs/2404.13076

Rendle, S., Zhang, L., and Koren, Y. (2019). "On the Difficulty of Evaluating Baselines: A Study on Recommender Systems." arXiv preprint arXiv:1905.01395. https://arxiv.org/abs/1905.01395

Roelofs, R., Fridovich-Keil, S., Miller, J., Shankar, V., Hardt, M., Recht, B., and Schmidt, L. (2019). "A Meta-Analysis of Overfitting in Machine Learning." Advances in Neural Information Processing Systems 32 (NeurIPS 2019). https://proceedings.neurips.cc/paper/2019/hash/ee39e503b6bedf0c98c388b7e8589aca-Abstract.html

Schluntz, E., and Zhang, B. (2024). "Building Effective Agents." Anthropic Engineering. https://www.anthropic.com/engineering/building-effective-agents

Si, C., Hashimoto, T., and Yang, D. (2026). "The Ideation-Execution Gap: Execution Outcomes of LLM-Generated versus Human Research Ideas." International Conference on Learning Representations (ICLR 2026). https://arxiv.org/abs/2506.20803

Song, Q., Cheng, D., Zhou, H., Yang, J., Tian, Y., and Hu, X. (2020). "Towards Automated Neural Interaction Discovery for Click-Through Rate Prediction." Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. ACM. https://doi.org/10.1145/3394486.3403137

Von Arx, S., Chan, L., and Barnes, E. (2025). "Recent Frontier Models Are Reward Hacking." METR. https://metr.org/blog/2025-06-05-recent-reward-hacking/

Wang, H., Wu, Y., Chang, D., Wei, L., and Heldt, L. (2026). "Self-Evolving Recommendation System: End-To-End Autonomous Model Optimization With LLM Agents." Proceedings of the 20th ACM Conference on Recommender Systems. ACM. https://doi.org/10.1145/3773078.3831919

Yan, W. (2026). "Multi-Agents: What's Actually Working." Cognition. https://cognition.com/blog/multi-agents-working

Zhang, M. R., Desai, N., Bae, J., Lorraine, J., and Ba, J. (2023). "Using Large Language Models for Hyperparameter Optimization." arXiv preprint arXiv:2312.04528. https://arxiv.org/abs/2312.04528